OOPS CONCEPTS

Object-Oriented Programming in Java

1. Introduction

Object-Oriented Programming (OOP) is a programming approach where a program is designed using objects and classes. Java mainly uses OOP concepts to make programs easier to understand, organize, reuse, and maintain.

The important OOP concepts are:

- Class
- Object
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

2. Class

My Understanding

A class is like a blueprint or design used to create objects. It contains variables and methods that describe the properties and actions of an object.

Real-World Example

A student record can be considered as a class. The class can contain details such as the student's name and a method to display the name.

Java Example

```
class Student {
    String name;

    void display() {
        System.out.println("Name: " + name);
    }
}
```

Key Point: A class is a blueprint used to create objects.

3. Object

My Understanding

An object is an instance of a class. It is created from the class and can access the variables and methods defined inside the class.

Real-World Example

If Student is a class, a particular student can be represented by creating an object of the Student class.

Java Example

```
class Student {
    String name;
```

```
    void display() {
        System.out.println("Name: " + name);
    }
}

public class ObjectExample {
    public static void main(String[] args) {
        Student student1 = new Student();

        student1.name = "Keerthana";
        student1.display();
    }
}
```

Key Point: An object is an instance of a class.

4. Encapsulation

My Understanding

Encapsulation means keeping the data inside a class protected and controlling how it can be accessed or changed. In Java, access modifiers such as private are commonly used for this purpose.

Real-World Example

In a bank account, the balance should not be changed directly by anyone. Instead, methods such as deposit can be used to update the balance.

Java Example

```
class BankAccount {
    private int balance = 1000;

    void deposit(int amount) {
        balance = balance + amount;
    }

    void displayBalance() {
        System.out.println("Balance: " + balance);
    }
}

public class EncapsulationExample {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        account.deposit(500);
        account.displayBalance();
    }
}
```

Key Point: Encapsulation protects data and controls access to it.

5. Inheritance

My Understanding

Inheritance allows one class to use the properties and methods of another class. It helps in reusing existing code instead of writing the same code again.

Real-World Example

A dog is an animal. Therefore, a Dog class can inherit common features from an Animal class and also have its own features.

Java Example

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

public class InheritanceExample {
    public static void main(String[] args) {
        Dog dog = new Dog();

        dog.eat();
        dog.bark();
    }
}
```

Key Point: Inheritance creates a parent-child relationship between classes.

6. Polymorphism

My Understanding

Polymorphism means "many forms". It allows the same method name or operation to behave differently depending on the situation.

In Java, polymorphism can mainly be achieved through:

- Method Overloading
- Method Overriding

6.1 Method Overloading

My Understanding

Method overloading occurs when two or more methods have the same name but different parameters.

Real-World Example

A calculator can perform addition with two numbers as well as three numbers. The method name can remain the same, but the number of parameters can be different.

Java Example

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class MethodOverloadingExample {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println(calc.add(10, 20));
        System.out.println(calc.add(10, 20, 30));
    }
}
```

Key Point: Method overloading uses the same method name with different parameters.

6.2 Method Overriding

My Understanding

Method overriding occurs when a child class provides its own version of a method that is already present in the parent class.

Real-World Example

An animal can make a sound, but different animals make different sounds. A Dog class can give its own implementation of the sound() method.

Java Example

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class MethodOverridingExample {
    public static void main(String[] args) {
        Dog dog = new Dog();

        dog.sound();
    }
}
```

Key Point: Method overriding allows a child class to provide its own behavior.

7. Abstraction

My Understanding

Abstraction means showing only the important information and hiding the internal implementation. In Java, abstraction can be achieved using abstract classes and interfaces.

Real-World Example

When we start a car, we only need to use the starting mechanism. We do not need to know all the internal processes happening inside the engine.

Java Example

```
abstract class Vehicle {
    abstract void start();
}

class Car extends Vehicle {
    void start() {
        System.out.println("Car starts with a key");
    }
}

public class AbstractionExample {
    public static void main(String[] args) {
        Car car = new Car();

        car.start();
    }
}
```

Key Point: Abstraction focuses on what an object does rather than how it does it.

8. Summary

OOP Concept	Simple Meaning
Class	Blueprint used to create objects
Object	Instance of a class
Encapsulation	Protecting data and controlling access
Inheritance	Reusing features from another class
Polymorphism	One name having different forms
Abstraction	Showing important details and hiding implementation

9. Conclusion

OOP concepts help us to write Java programs in a structured and organized way. Class and object are the basic building blocks of OOP. Encapsulation helps to protect data, inheritance helps to reuse code, polymorphism allows different forms of behavior, and abstraction hides unnecessary implementation details.

By using these concepts, Java programs become easier to understand, maintain, and reuse.